

Artificial Intelligence

SOLVING
PROBLEMS BY
SEARCHING





Artificial Intelligence - Solving Problems by Searching

Search: In Artificial Intelligence (AI), search is the process of exploring different possible solutions to a problem in order to find the best or most optimal solution.

- It is like problem-solving where an AI agent moves step-by-step through a set of possible actions to reach a goal state.

Search Concepts

Search: Searching is a **step-by-step** procedure to solve a **search-problem** in a **given search space**.

A **search problem** can have **three main factors**:

Search Space: Search space represents a **set of possible solutions**, which a system may have

Start State: It is a state from where **agent begins the search**.

Goal test: It is a **function** which observe the **current state** and **returns** whether the **goal state is achieved or not**.

Uninformed Search and Informed Search

- ➡ **Uninformed Search and Informed Search**
Two fundamental approaches in **artificial intelligence (AI)** and **computer science** for solving problems using **search algorithms**. They differ in how they explore the search space to find a solution.

1. Uninformed Search (Blind Search)

- “Uninformed search algorithms do not use any problem-specific knowledge or heuristics to guide the search. They explore the search space blindly, relying only on the information available in the problem definition (e.g., the start state, goal state, and actions)”.
- means that these algorithms explore the search space without any **additional information** about the problem beyond **the basic definition** (e.g., the **start state, goal state, and actions**). They rely solely on the structure of the search space and do not take advantage of any domain-specific insights or estimates to prioritize or direct the search.

Characteristics

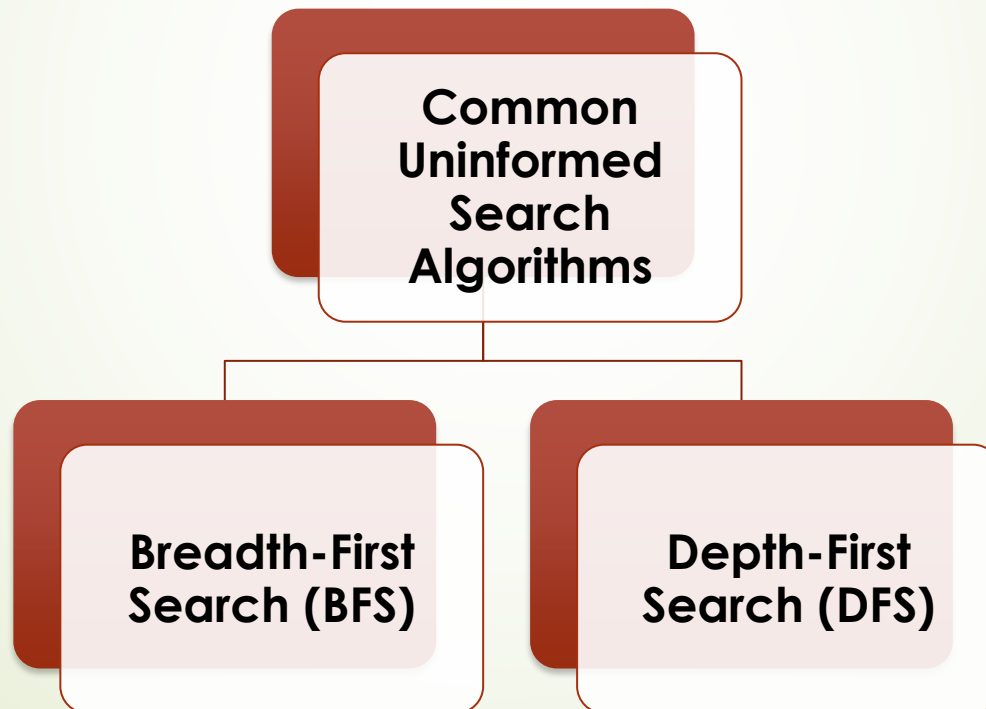


No additional information about the problem is used.

Explores all possible paths systematically.

Often less efficient for large search spaces.

Common Uninformed Search Algorithms



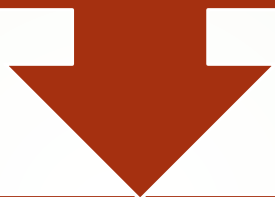


Example: Finding a Book in a Library

- Imagine you are in a large library with thousands of books, and you need to find a specific book (e.g., *"Introduction to Algorithms"*). However, you don't have access to the library's catalog, maps, or any information about where the book might be located. You also don't know how the books are organized (e.g., by genre, author, or title). This is similar to how **uninformed search algorithms** work—they have no additional information to guide their search



How Uninformed Search Works in This Scenario



Since you don't have any problem-specific knowledge (e.g., the book's location or the library's organization system), you have to rely on a systematic but blind approach to find the book. Here's how different uninformed search strategies might work:



1. Breadth-First Search (BFS)

How it works: You start at the entrance of the library and explore every shelf on the first floor before moving to the second floor, then the third floor, and so on.

Behavior: You methodically check every shelf in order, ensuring you don't miss any section.

Pros: You are guaranteed to find the book if it's in the library.

Cons: It's time-consuming because you might check many irrelevant shelves before finding the book.



2. Depth-First Search (DFS):

How it works: You start at the entrance, pick a random shelf, and start searching through every book on that shelf. If you don't find the book, you move to the next shelf in the same section. You keep going deeper into one section before backtracking to explore other sections.

Behavior: You might spend a lot of time searching through one section of the library before realizing the book isn't there.

Pros: It uses less memory than BFS because you don't need to keep track of all the shelves you've visited.

Cons: You might get stuck searching in the wrong section for a long time, especially if the book is in a different part of the library.



2. Informed Search (Heuristic Search)

- ➔ Informed search algorithms use **problem-specific knowledge**, often in the form of a **heuristic function**, to guide the search toward the **goal more efficiently**. The heuristic function estimates **the cost to reach** the goal from a given state, helping the algorithm prioritize which paths to explore.



Characteristics:

- Uses domain-specific knowledge to improve efficiency.
- Often faster and more memory-efficient than uninformed search.
- Heuristics must be carefully designed to ensure optimality and completeness.

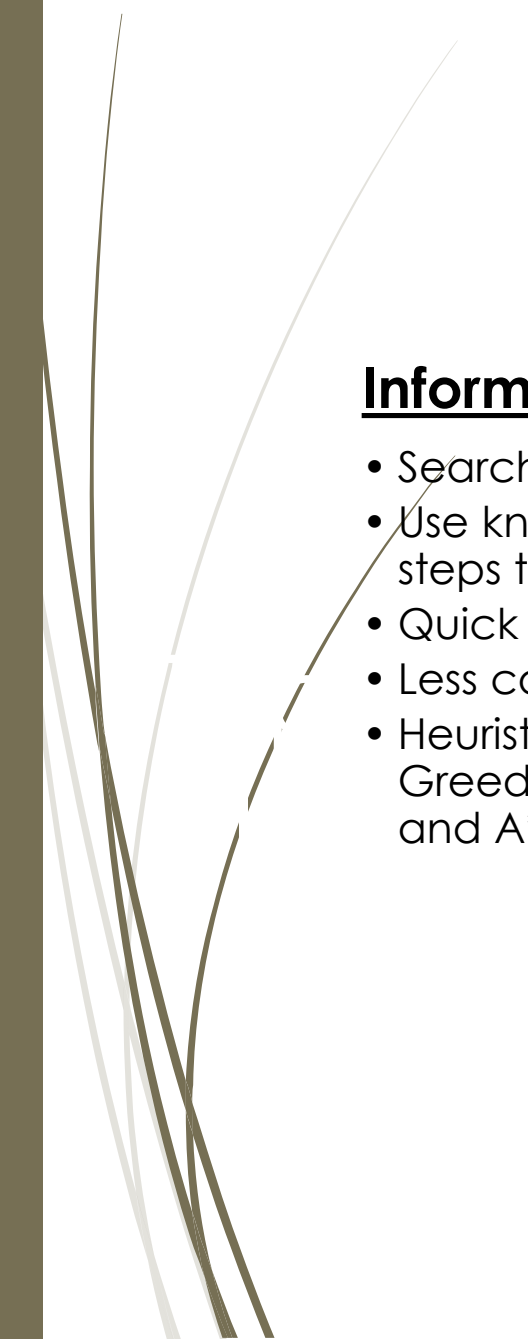


Common Informed Search Algorithms:

Common Informed Search Algorithms:

Greedy Best-First Search:

*A Search:**



Informed Search

- Search with information
- Use knowledge to find steps to solution
- Quick solution
- Less complexity
- Heuristic Function, Greedy Best First Search, and A* search.

Uninformed Search

- Search without information
- No knowledge
- Time consuming
- More complexity (time, space)
- Depth First Search (DFS), Breadth First Search (BFS)

Heuristics function:

- Heuristic is a function which is used in **Informed Search**, and it finds the most promising path. It takes the **current state** of the **agent as its input** and **produces the estimation of how close agent is from the goal**. The heuristic method, however, might not always give the best solution, but it is guaranteed to find a good solution in reasonable time. **Heuristic function estimates how close a state is to the goal**. It is represented by $h(n)$, and it calculates the cost of an optimal path between the pair of states. **The value of the heuristic function is always positive.**
- $h(n) \leq h^*(n)$
- Here $h(n)$ is heuristic cost, and $h^*(n)$ is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Breadth First Search

➤ Breadth-First Search (BFS) is a fundamental algorithm used for traversing or **searching tree or graph data structures**. It explores all the **nodes at the present depth level** before moving on to nodes at the **next depth level**. This makes BFS particularly useful for finding the shortest path in **unweighted graphs**.

➤ **Key Points to Understand BFS**

1. Traversal Strategy:

1. BFS explores nodes level by level.
2. It starts at the root (or any arbitrary node in a graph) and explores all its neighbors before moving to the next level of nodes.



Key Points to Understand BFS

2. Data Structure Used:

1. BFS uses a **Queue** (First-In-First-Out, FIFO) to keep track of nodes to be explored.
2. The queue ensures that nodes are processed in the order they are discovered.

3. Visited Nodes:

1. To avoid revisiting nodes, BFS maintains a list or set of **visited nodes**.

4. **Graph Representation:** Can be used with both adjacency lists and matrices.

- **Tree Traversal:** Used for level-order traversal in trees.
- **Backtracking:** No backtracking



Key Points to Understand BFS

- Algorithm Type: Greedy approach (explores level by level).
- Graph Type: Applies to both directed and undirected graphs
- Time Complexity: $O(V + E)$
- Space Complexity: $O(V)$ (due to queue storing nodes at the current level)



BFS algorithm step by step:

1. Initialization:

- Choose a starting node (root).
- Create an empty queue and add the starting node to it.
- Create a set or list to keep track of visited nodes and mark the starting node as visited.

2. Exploration:

- While the queue is not empty:
 - Dequeue the front node from the queue.
 - Process the node (e.g., print it or check if it's the target).
 - Enqueue all unvisited neighbors of the dequeued node and mark them as visited.

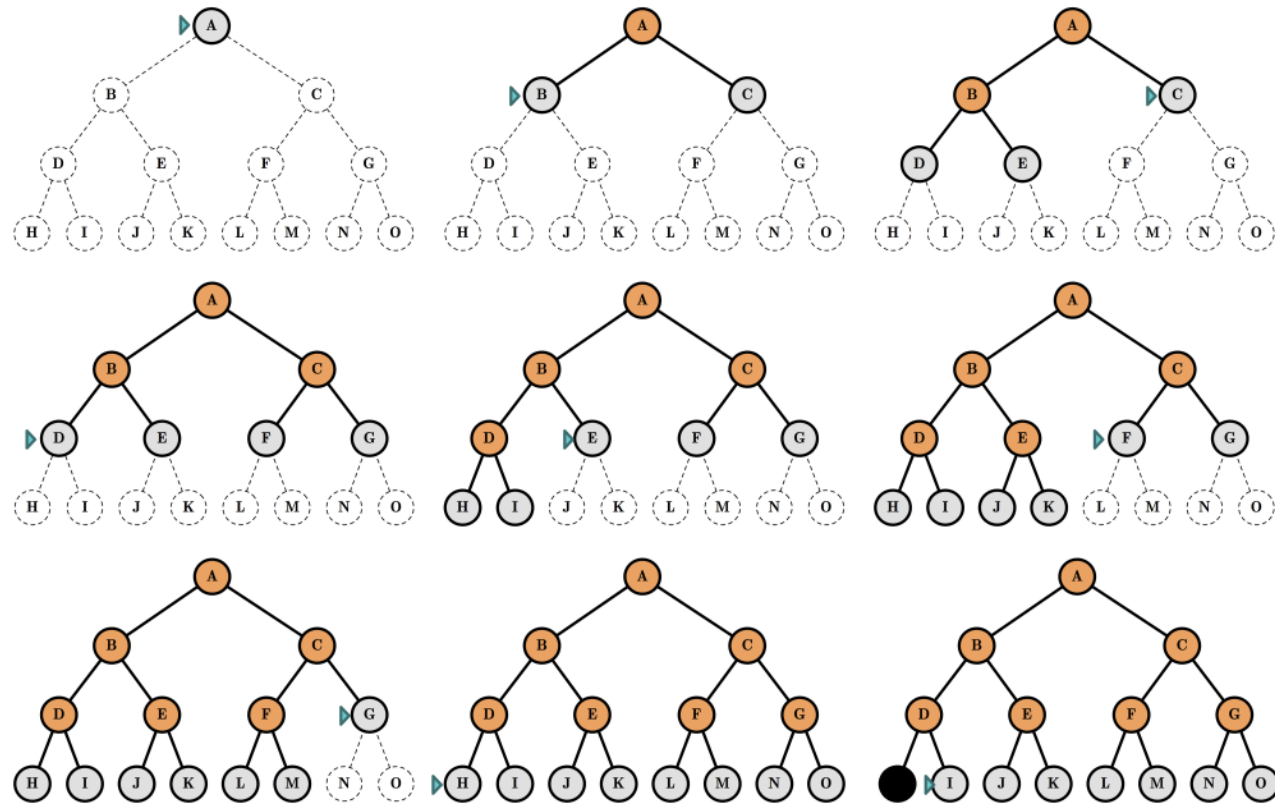


BFS algorithm step by step:

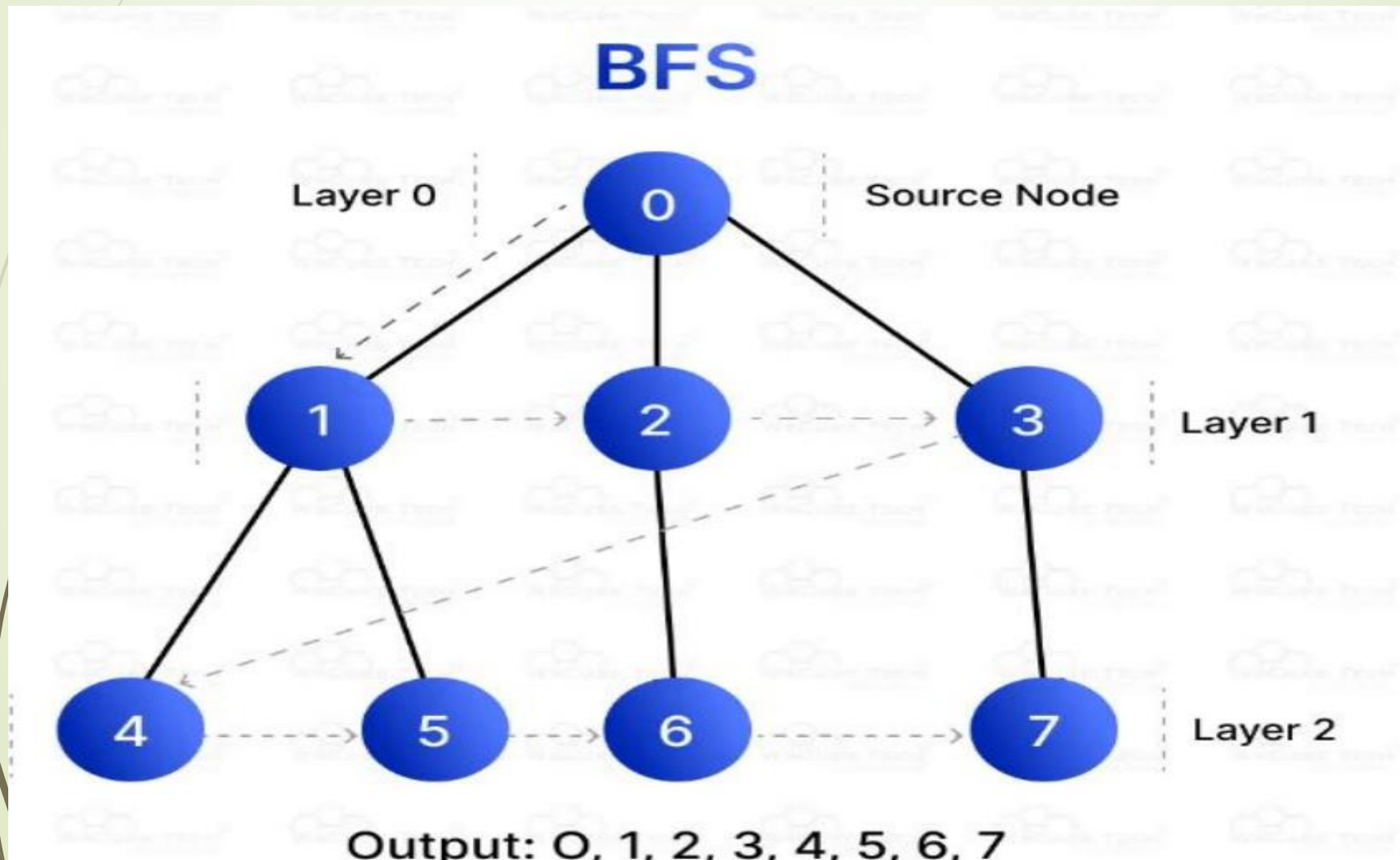
3. Termination:

- The algorithm terminates when the queue is empty, meaning all reachable nodes have been visited.

BFS: Expand shallowest first.



Example# 1





BFS (Example 1)

Traversal Order:

- Starting from node 0:
- Layer 0: Visit node 0 (source node).
- Layer 1: Visit nodes 1, 2, 3 (neighbors of node 0).
- Layer 2: Visit nodes 4, 5, 6, 7 (neighbors of nodes 1, 2, 3).
- Output: 0, 1, 2, 3, 4, 5, 6, 7.

Example#2 of BFS

A

/ \

B C

/ \ \

D E F



Example#2 of BFS

Step-by-Step Execution:

1. Start at node **A**.
 1. Queue: [A]
 2. Visited: {A}
2. Dequeue **A** and process it.
 1. Enqueue its neighbors: **B** and **C**.
 2. Queue: [B, C]
 3. Visited: {A, B, C}
3. Dequeue **B** and process it.
 1. Enqueue its neighbors: **D** and **E**.
 2. Queue: [C, D, E]
 3. Visited: {A, B, C, D, E}
4. Dequeue **C** and process it.
 1. Enqueue its neighbor: **F**.
 2. Queue: [D, E, F]
 3. Visited: {A, B, C, D, E, F}



Example#2 of BFS

1. Dequeue **D** and process it.
 1. No unvisited neighbors.
 2. Queue: [E, F]
 3. Visited: {A, B, C, D, E, F}
 2. Dequeue **E** and process it.
 1. No unvisited neighbors.
 2. Queue: [F]
 3. Visited: {A, B, C, D, E, F}
 3. Dequeue **F** and process it.
 1. No unvisited neighbors.
 2. Queue: []
 3. Visited: {A, B, C, D, E, F}
- ➡ The traversal order is: **A → B → C → D → E → F.**



BFS

pseudocode

- `def BFS(graph, start):`
- `queue = [start] # Initialize queue with the starting node`
- `visited = set() # Set to keep track of visited nodes`
- `visited.add(start)`
- `while queue:`
- `node = queue.pop(0) # Dequeue the front node`
- `print(node) # Process the node (e.g., print it)`
- `for neighbor in graph[node]: # Explore neighbors`
- `if neighbor not in visited:`
- `visited.add(neighbor)`
- `queue.append(neighbor) # Enqueue unvisited neighbors`



1. def BFS(graph, start):

This defines a function named BFS that takes two arguments:

- graph: The graph represented as an adjacency list (a dictionary where keys are nodes and values are lists of neighbors).
- start: The starting node from which the BFS traversal begins.



2 queue = [start]

- A queue is initialized with the start node. The queue will be used to keep track of nodes to be explored.
- The queue follows the **First-In-First-Out (FIFO)** principle, meaning the first node added will be the first one removed.



3.visited = set()

- A **set** named **visited** is created to keep track of nodes that have already been visited.

4. **visited.add(start)**

- The **start** node is added to the **visited** set because it is the first node being processed.



5. while queue:

- This starts a loop that continues as long as the queue is not empty.
- The loop ensures that all nodes in the graph (reachable from the start node) are processed.



**6. node =
queue.pop(0)**

- The first node in the queue is removed (dequeued) and stored in the variable node.
- pop(0) removes the element at index 0, which is the front of the queue (FIFO behavior)



7. `print(node)`

- The node is processed. In this case, it is printed, but you could perform other operations here (e.g., checking if it's the target node).



8. **for neighbor in graph[node]:**

- This loop iterates over all the neighbors of the current node.
- The `graph[node]` retrieves the list of neighbors for the current node from the adjacency list.



9. if neighbor not in visited:

- This checks if the neighbor has not been visited yet.
- If the neighbor is not in the visited set, it means it hasn't been processed yet.



10. `visited.add(neighbor)`

The `neighbor` is added to the `visited` set to mark it as visited

11. `queue.append(neighbor)`

- The `neighbor` is added to the end of the queue so that it can be processed in the next iterations of the `while` loop.



12. End of the function

- The loop continues until the queue is empty, meaning all reachable nodes have been processed.
- The function then terminates.



Advantages of BFS

1. Guarantees the shortest path in unweighted graphs.
2. Works well for finding connected components.
3. Easy to implement using a queue.

Disadvantages of BFS

1. Requires more memory compared to Depth-First Search (DFS) for wide graphs.
2. Not suitable for large graphs due to memory constraints.



Depth-First Search (DFS)

- Depth-First Search (DFS) is another fundamental algorithm used for traversing or searching tree or graph data structures. Unlike Breadth-First Search (BFS), which explores nodes level by level, DFS explores as far as possible along each branch before backtracking. This makes DFS particularly useful for solving problems like finding connected components, detecting cycles, and solving puzzles.
- Time Complexity: $O(V + E)$
- • Graph Type: Applies to both directed and undirected graphs
- • Traversal Method: Explores as deep as possible along one branch before backtracking.
- • Data Structure Used: Stack (LIFO) or recursion stack.



Depth-First Search (DFS)

Space Complexity: $O(V)$ (due to recursion or explicit stack)

- **Shortest Path:** Not guaranteed to find the shortest path but explores all paths.
- **Pathfinding in Weighted Graphs:** Not suitable for weighted graphs.
- **Recursive/Iterative:** Can be recursive (via recursion stack) or iterative (using an explicit stack).

Graph Representation: Can be used with both adjacency lists and matrices.

- **Tree Traversal:** Used for pre-order, in-order, and post-order traversal in trees.
- **Backtracking:** Uses backtracking to explore new branches.
- **Algorithm Type:** Backtracking approach (explores depth first).



Key Points to Understand DFS

1. Traversal Strategy:

1. DFS explores nodes by going deeper into the graph or tree.
2. It starts at the root (or any arbitrary node in a graph) and explores as far as possible along each branch before backtracking.

2. Data Structure Used:

1. DFS uses a **Stack** (Last-In-First-Out, LIFO) to keep track of nodes to be explored.
2. The stack can be implemented explicitly using a data structure or implicitly using recursion.

3. Visited Nodes:

1. To avoid revisiting nodes, DFS maintains a list or set of **visited nodes**.



Applications:

1. Finding connected components in a graph.
2. Detecting cycles in a graph.
3. Solving puzzles like mazes or the "N-Queens" problem.
4. Topological sorting of nodes in a directed acyclic graph (DAG).



DFS algorithm step by step:

1. Initialization:

- Choose a starting node (root).
- Create a set or list to keep track of visited nodes.

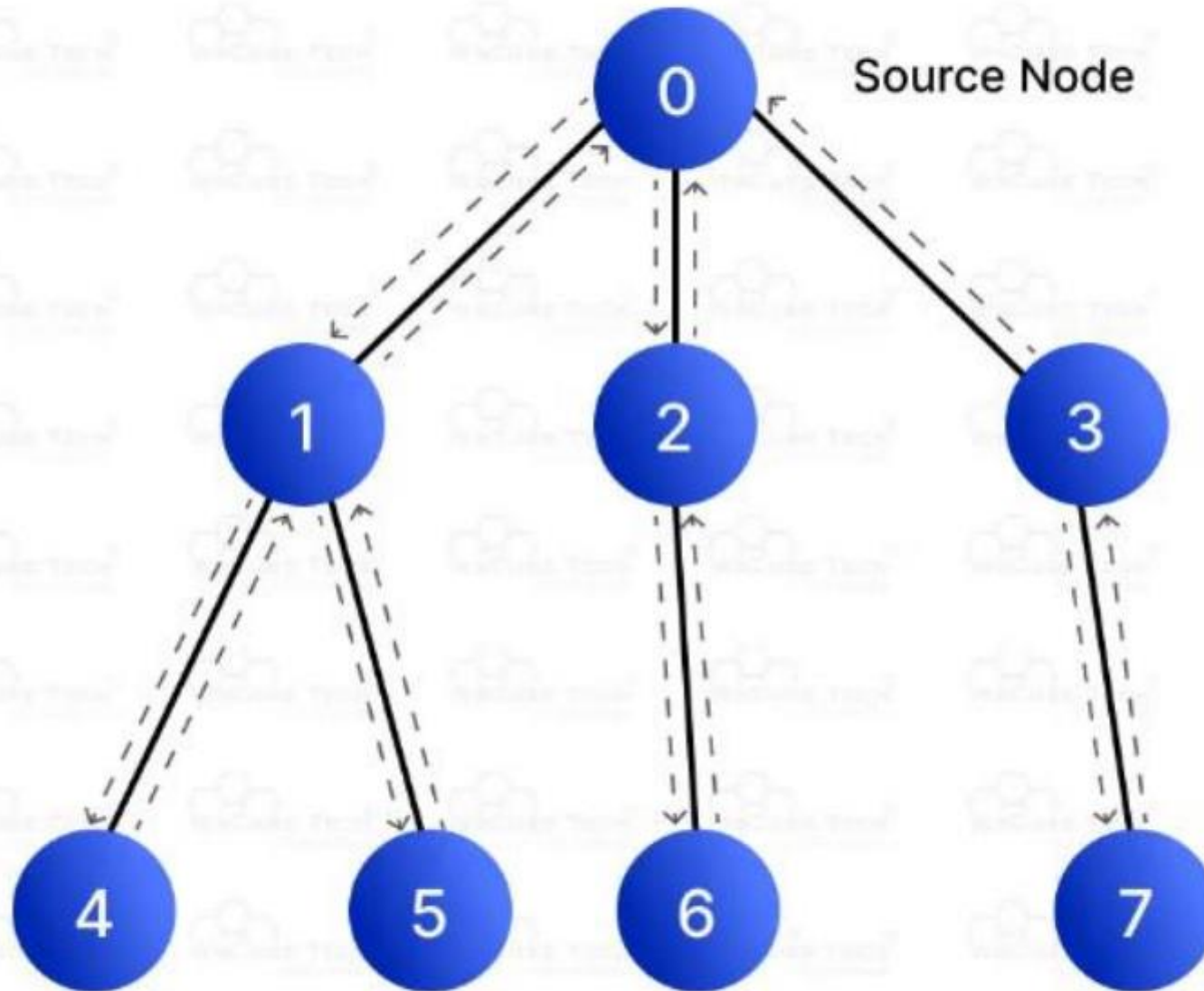
2. Exploration:

- Mark the current node as visited and process it.
- For each unvisited neighbor of the current node:
 - Recursively perform DFS on the neighbor.

3. Termination:

- The algorithm terminates when all reachable nodes have been visited.

DFS



Output: 0, 1, 4, 5, 2, 6, 3, 7

Example of DFS

A

/ \

B C

/ \ \

D E F

Step-by-Step Execution:

1

Start at node **A**.

- Mark A as visited.
- Process A (e.g., print it).

2

Explore neighbors of A: B and C.

- Visit B (first neighbor of A).
 - Mark B as visited.
 - Process B.



Explore neighbors of B: D and E.

- Visit D (first neighbor of B).
- Mark D as visited.
- Process D.
- D has no unvisited neighbors, so backtrack to B.

Visit E (second neighbor of B).

- Mark E as visited.
- Process E.
- E has no unvisited neighbors, so backtrack to B.

B has no more unvisited neighbors, so backtrack to A.



Visit C (second neighbor of A).

- Mark C as visited.
- Process C.

Explore neighbors of C: F.

- Visit F (first neighbor of C).
 - Mark F as visited.
 - Process F.
- F has no unvisited neighbors, so backtrack to C.

C has no more unvisited neighbors, so backtrack to A.

A has no more unvisited neighbors, so the algorithm terminates.

The traversal order is: **A** → **B** → **D** → **E** → **C** → **F**.

DFS

pseudocode

- `def DFS(graph, node, visited):`
- `if node not in visited:`
- `print(node) # Process the node`
 (e.g., print it)
- `visited.add(node) # Mark the`
 node as visited
- `for neighbor in graph[node]: #`
 Explore neighbors
- `DFS(graph, neighbor, visited) #`
 Recursive call

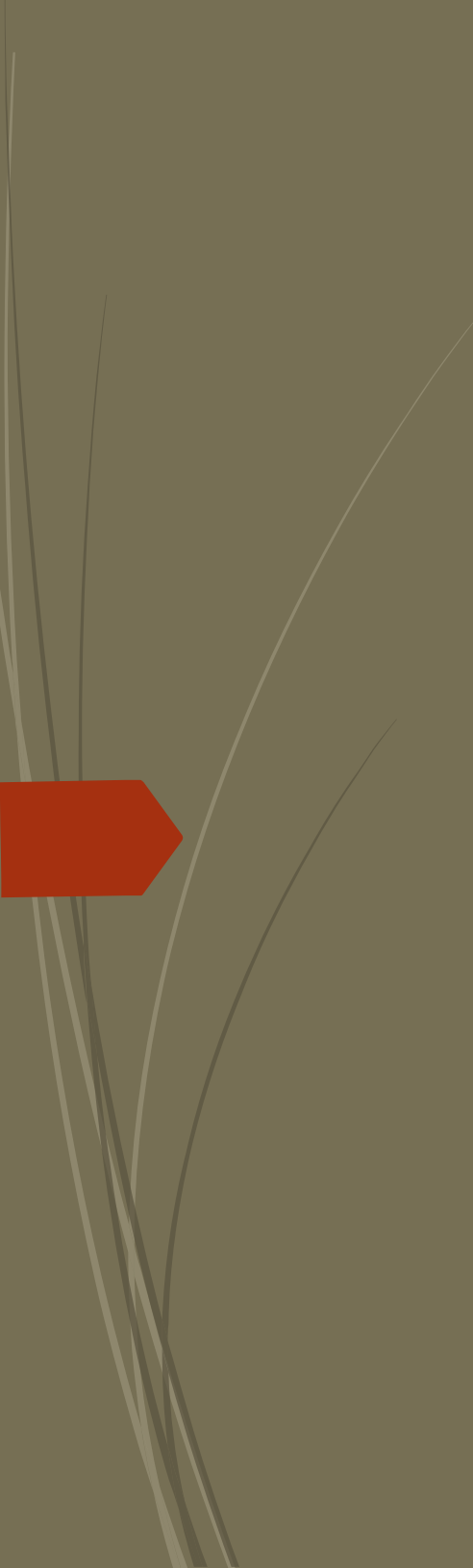


def DFS(graph, node, visited):

- This defines a function named **DFS** that takes three arguments:
 - **graph**: The graph represented as an adjacency list (a dictionary where keys are nodes and values are lists of neighbors).
 - **node**: The current node being processed.
 - **visited**: A set to keep track of visited nodes.

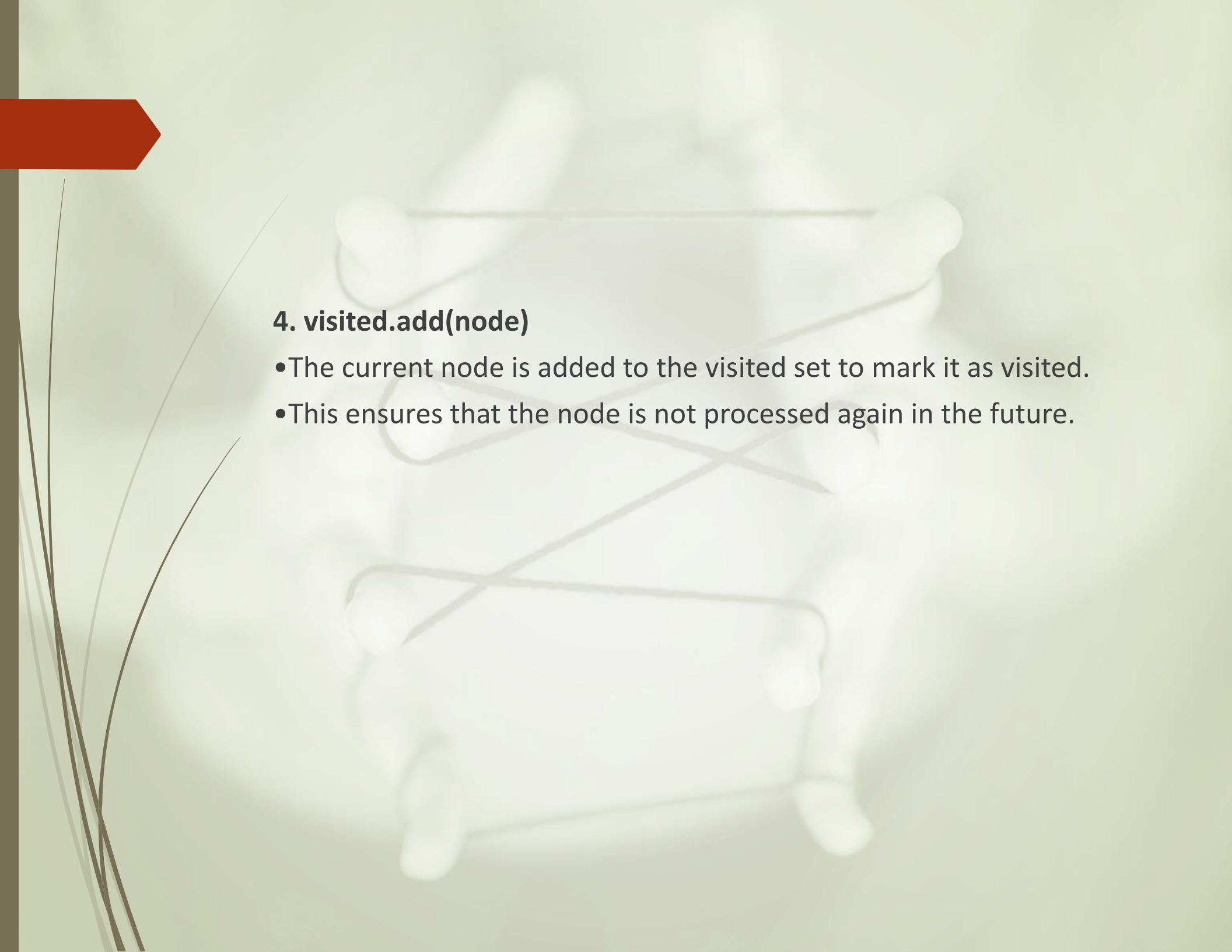
2. if node not in visited:

- This checks if the current node has not been visited yet.
- If the node is not in the visited set, it means it hasn't been processed yet, so we proceed to process it.



3. **print(node)**

- The current node is processed. In this case, it is printed, but you could perform other operations here (e.g., checking if it's the target node).



4. **visited.add(node)**

- The current node is added to the visited set to mark it as visited.
- This ensures that the node is not processed again in the future.

5. for neighbor in graph[node]:

- This loop iterates over all the neighbors of the current node.
- The `graph[node]` retrieves the list of neighbors for the current node from the adjacency list.



6. DFS(graph, neighbor, visited)

- This is a recursive call to the DFS function.
- It explores the neighbor node in the same way as the current node.
- The recursion ensures that DFS explores as far as possible along each branch before backtracking.

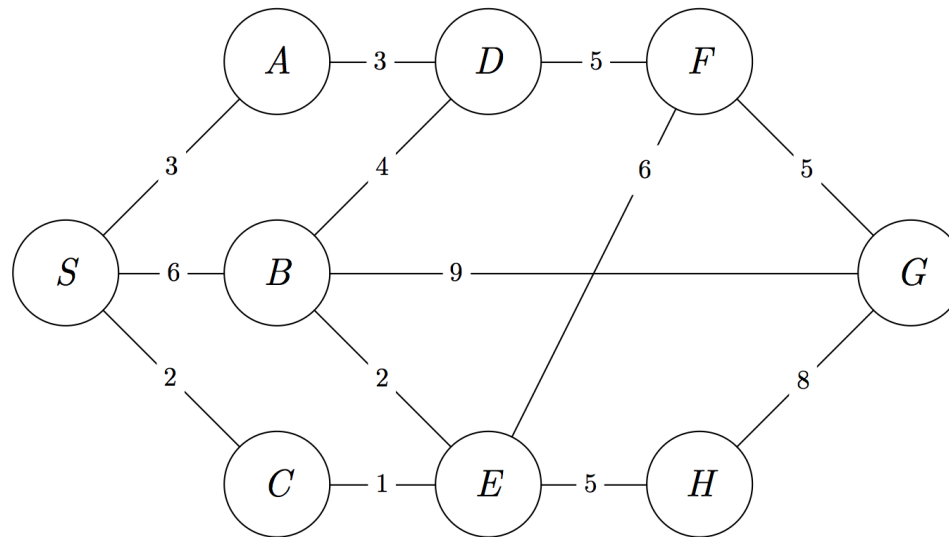
Advantages of DFS

1. Requires less memory than BFS for deep graphs (uses a stack).
2. Suitable for solving problems that require exploring all possible paths (e.g., mazes, puzzles).
3. Works well for finding connected components in a graph.
4. Efficient for detecting cycles in a graph.
5. Simple to implement using recursion.

Disadvantages of DFS

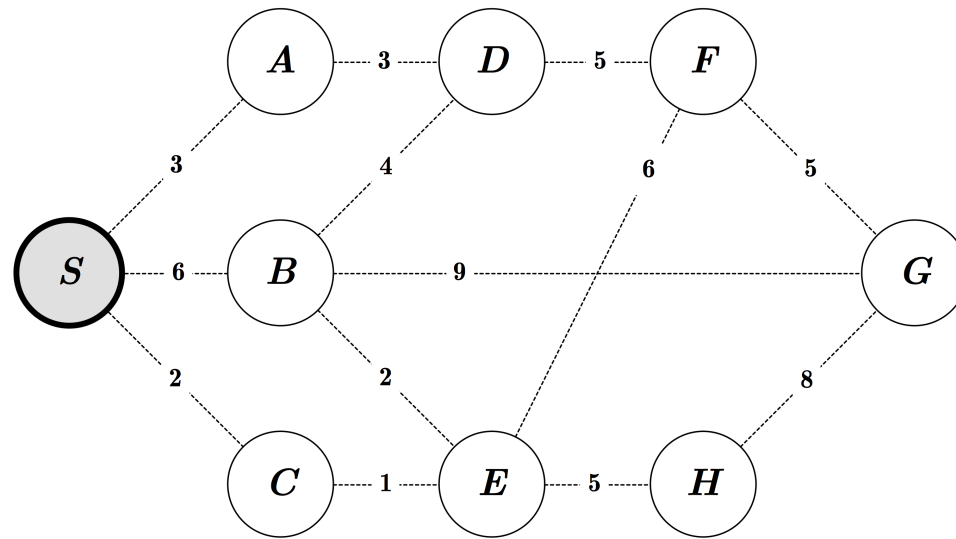
1. Does not guarantee the shortest path in unweighted graphs.
2. Can get stuck in infinite loops if cycles are not handled properly.
3. May not explore all nodes in disconnected graphs unless modified.
4. Recursive implementation can lead to stack overflow for very deep graphs.
5. Not ideal for finding the shortest path in unweighted graphs.

Exercise

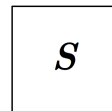


Question: What is the **order of visits of the nodes** and the **path** returned by BFS, DFS and UCS?

Exercise: BFS

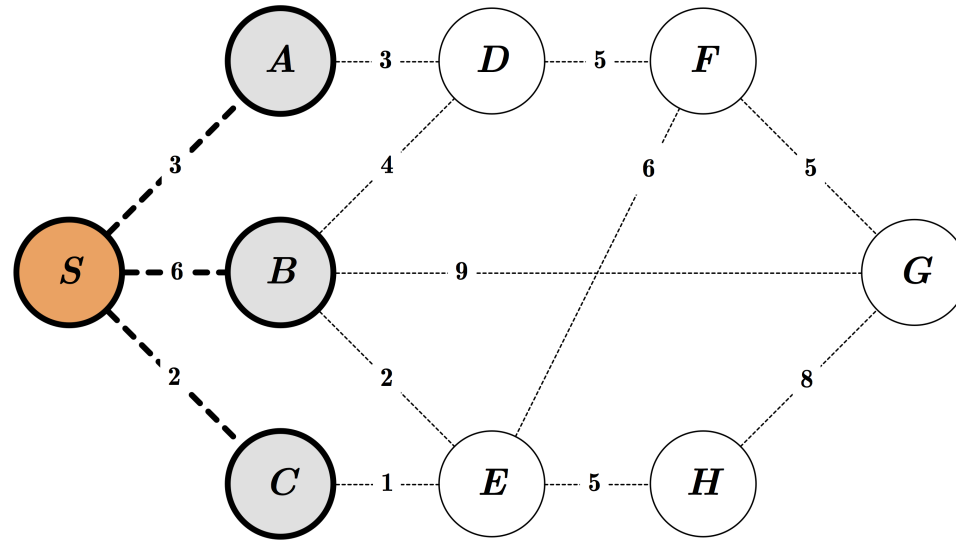


Queue:



Order of Visit:

Exercise: BFS



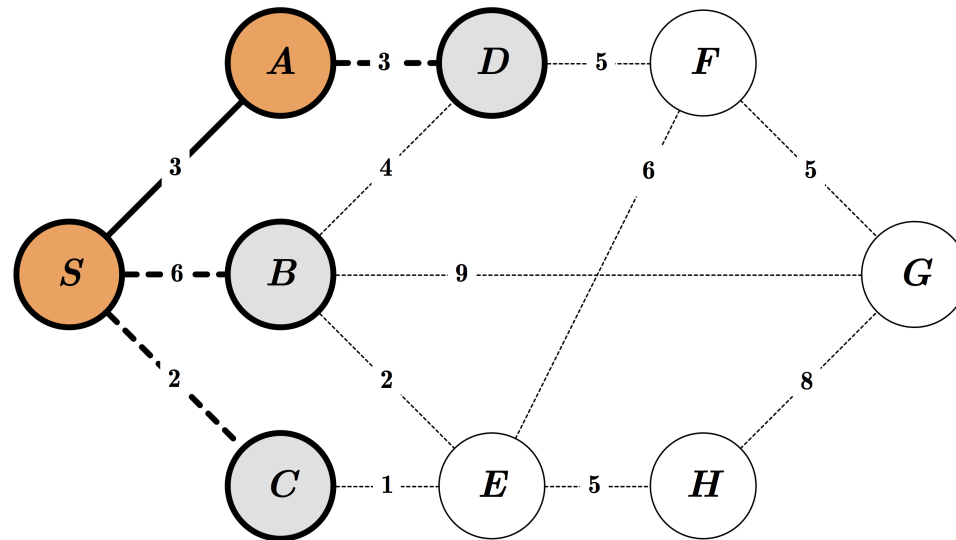
Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>
----------	----------	----------	----------

Order of Visit:

S

Exercise: BFS



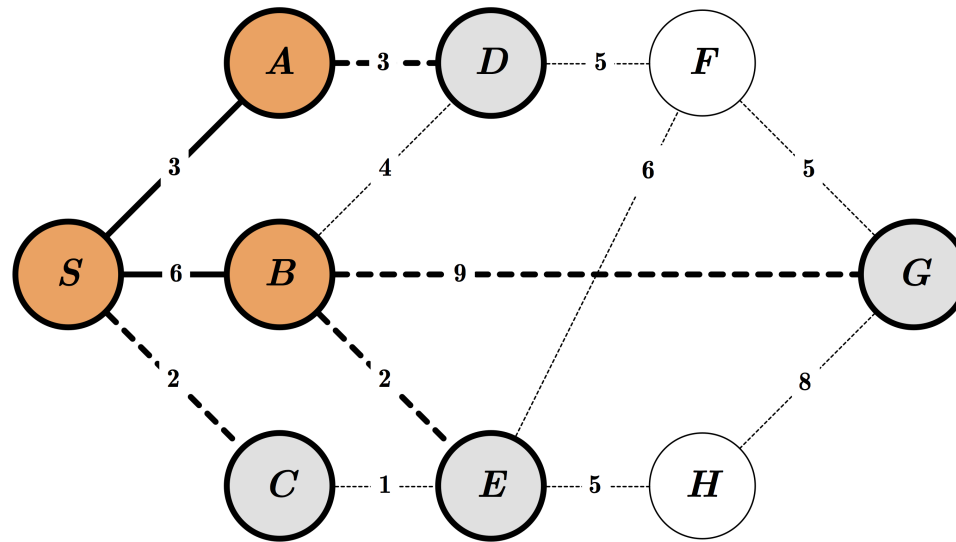
Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
----------	----------	----------	----------	----------

Order of Visit:

S *A*

Exercise: BFS



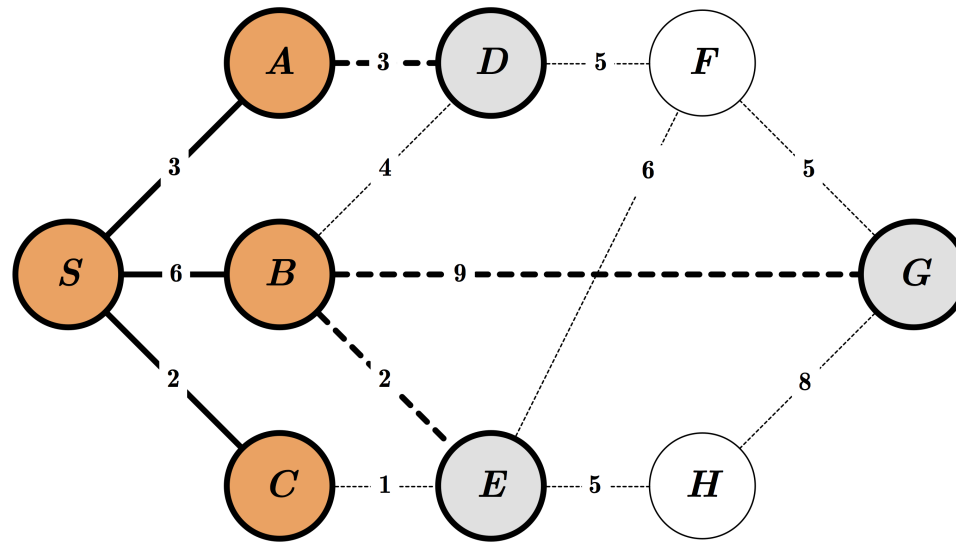
Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>G</i>
----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *B*

Exercise: BFS



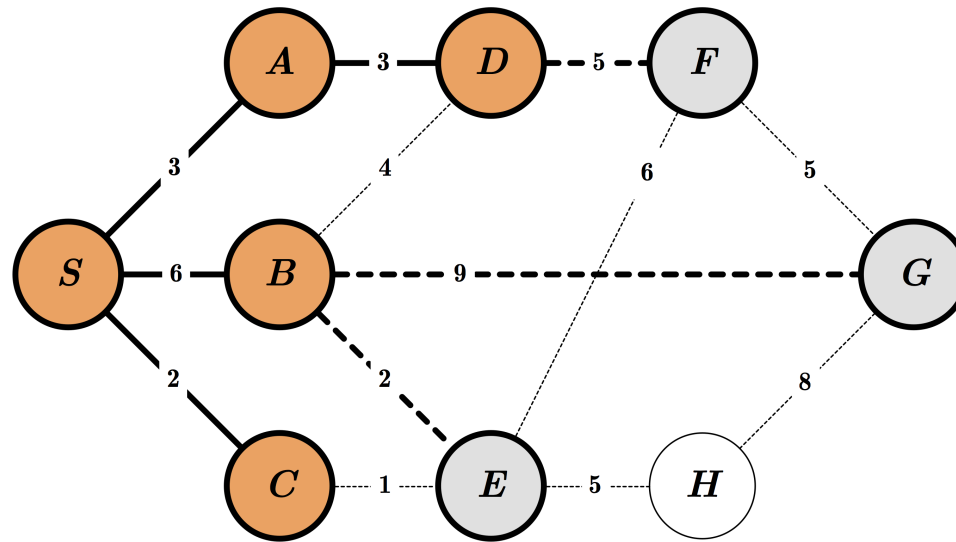
Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>G</i>
----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *B* *C*

Exercise: BFS



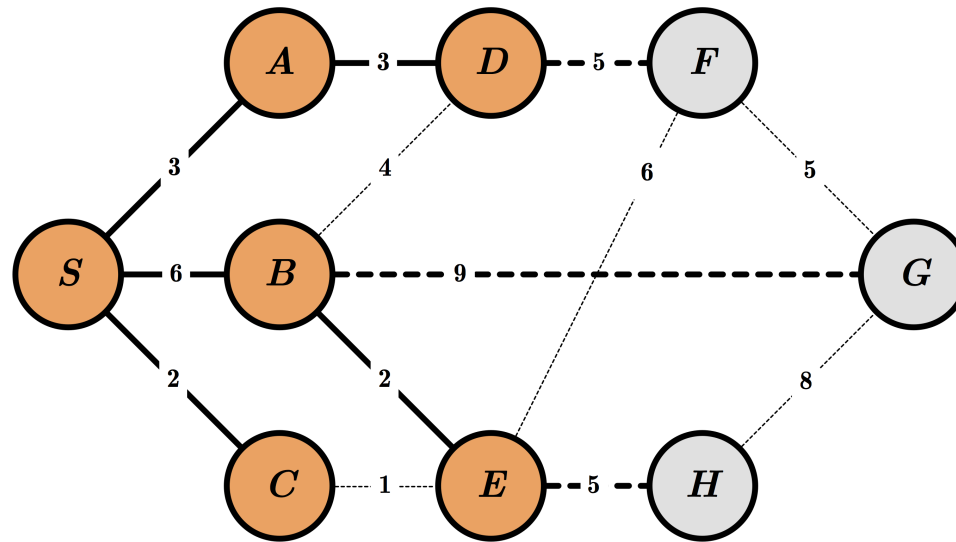
Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>G</i>	<i>F</i>
----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *B* *C* *D*

Exercise: BFS



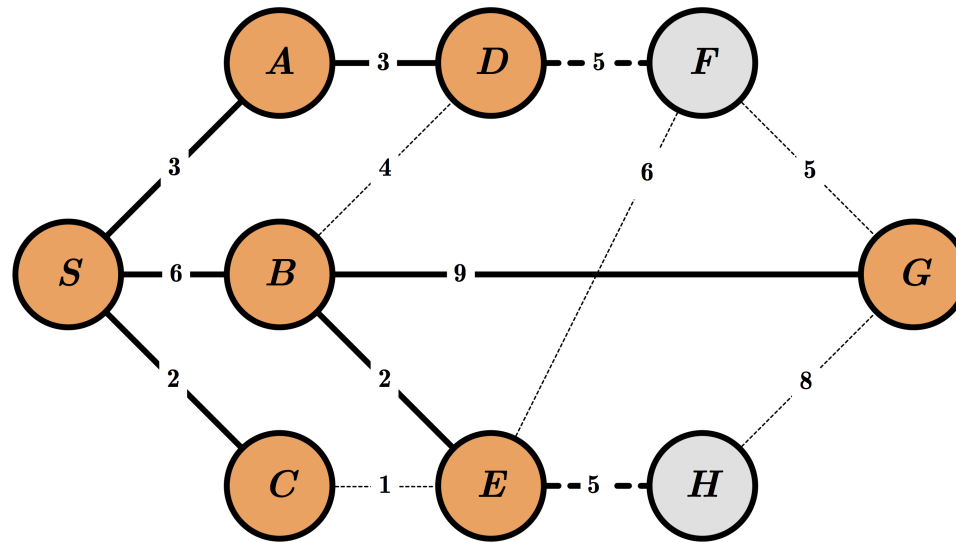
Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>G</i>	<i>F</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *B* *C* *D* *E*

Exercise: BFS



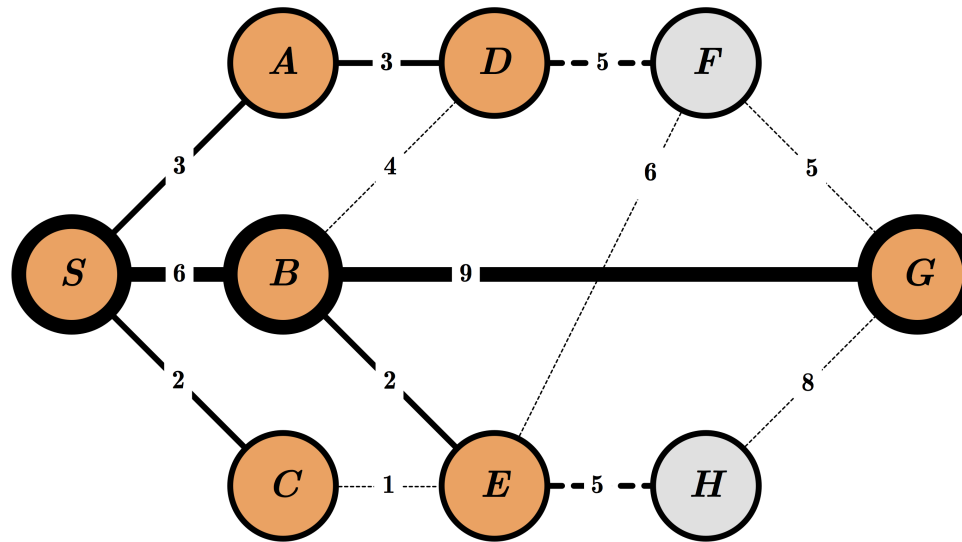
Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>G</i>	<i>F</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *B* *C* *D* *E* *G*

Exercise: BFS



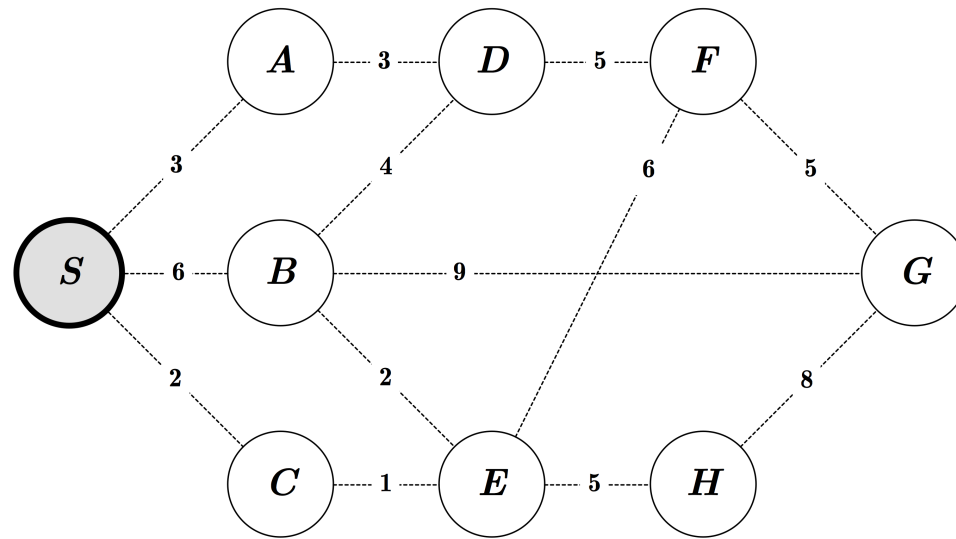
Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>G</i>	<i>F</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

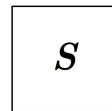
Order of Visit:

S *A* *B* *C* *D* *E* *G*

Exercise: DFS

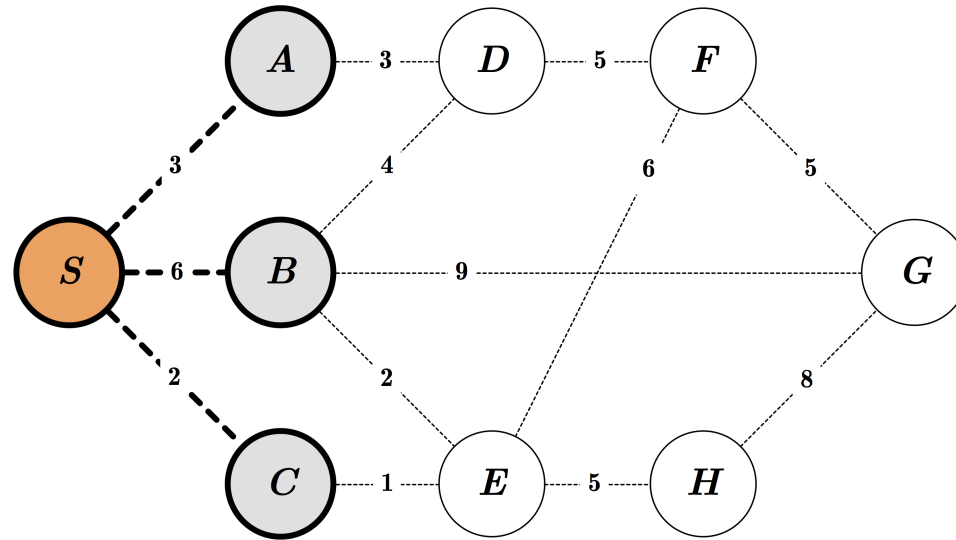


Stack:



Order of Visit:

Exercise: DFS



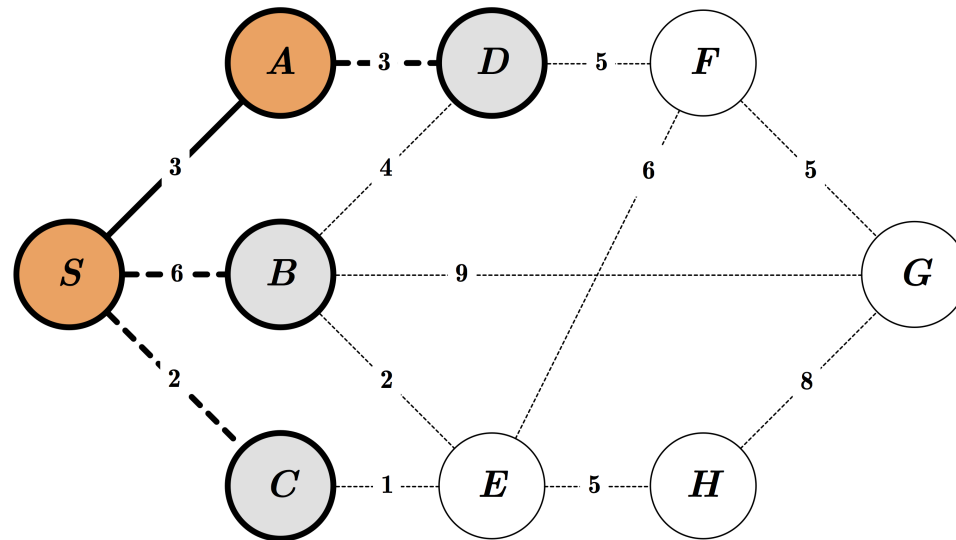
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>
----------	----------	----------	----------

Order of Visit:

S

Exercise: DFS



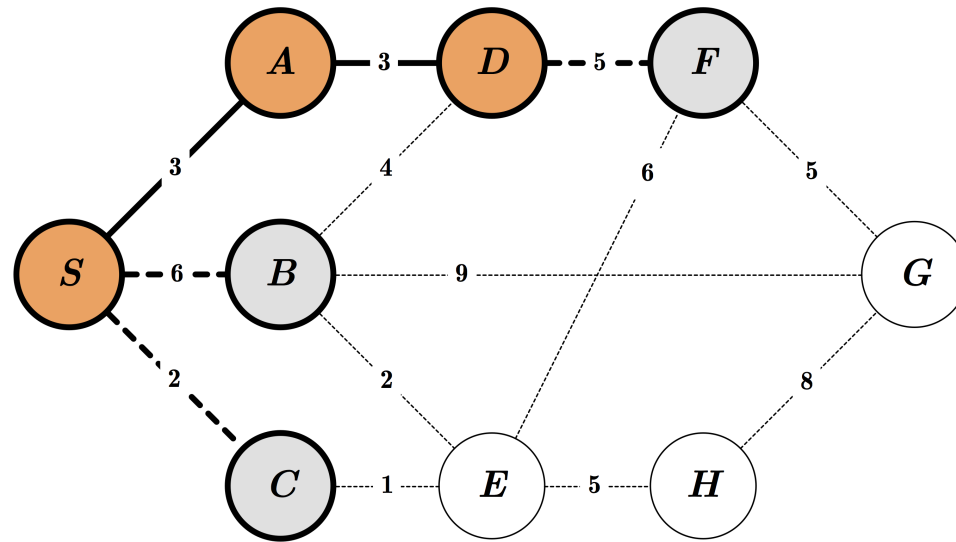
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>
----------	----------	----------	----------	----------

Order of Visit:

S *A*

Exercise: DFS



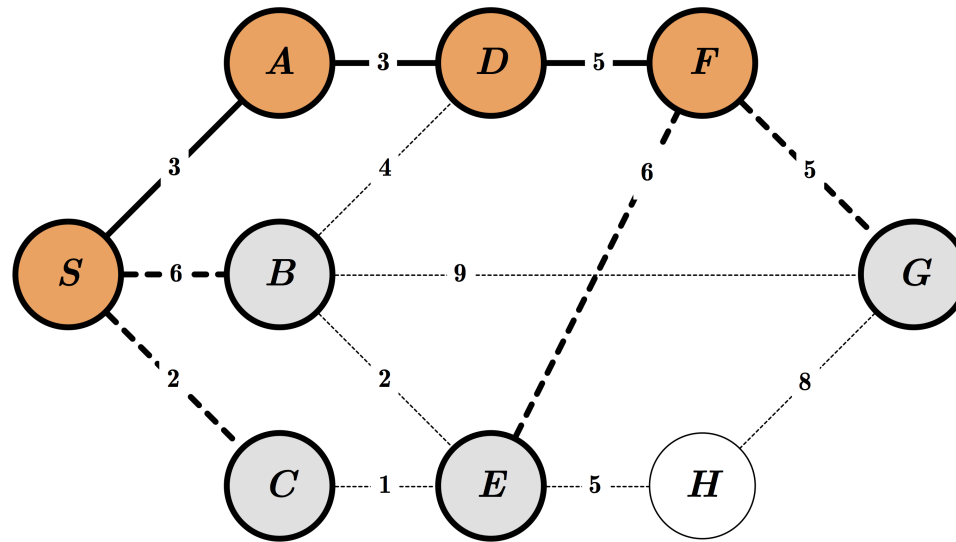
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>
----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D*

Exercise: DFS



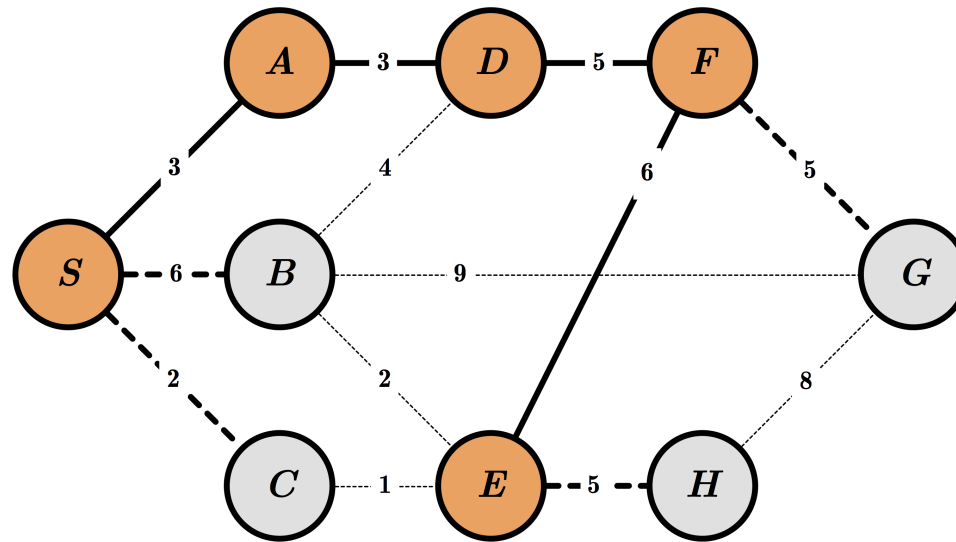
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>
----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F*

Exercise: DFS



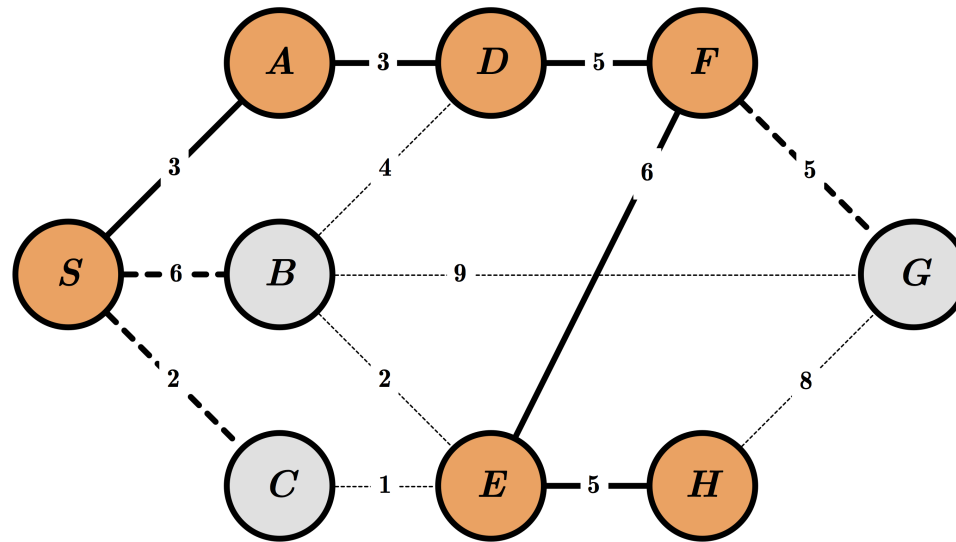
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F* *E*

Exercise: DFS



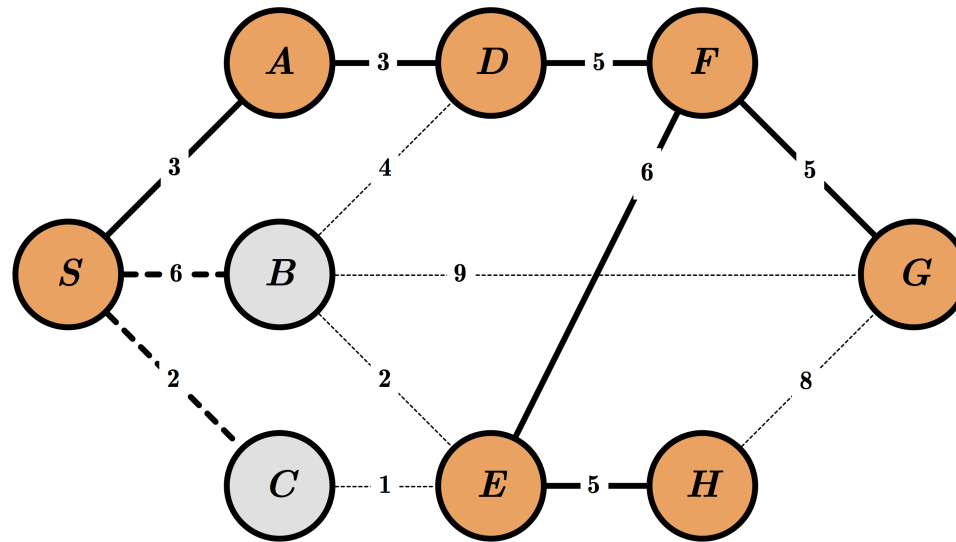
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F* *E* *H*

Exercise: DFS



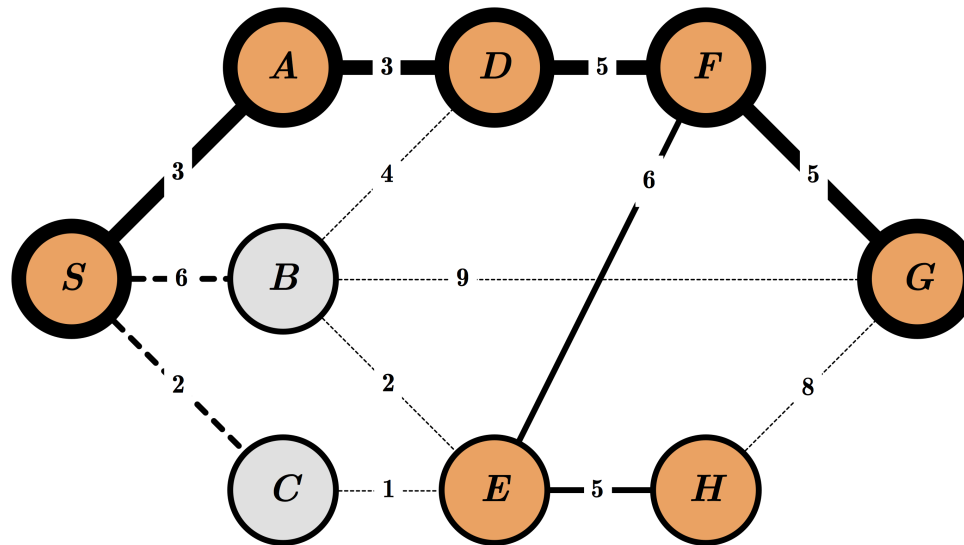
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F* *E* *H* *G*

Exercise: DFS



Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F* *E* *H* *G*